

Lab 02

This lab will walk you through an example on how to write and run an R function, the code for performing a randomization test and a self-assessment part.

How to set up and use a function in R. Example: Standard Error

We start by reading in our data, which is a .csv file whose first row contains the column names. By default, `header=TRUE`, but it's always wise to check this by opening the actual file and setting this parameter explicitly. Failing to properly set `header` can lead to some unexpected results.

```
# read in the data
library(tidyverse) # make sure to install if it isn't already
creativity <- read.csv("creativity.csv", header=TRUE)

# print the first 6 entries
head(creativity)
```

```
  treatment score
1 intrinsic  12.0
2 intrinsic  12.0
3 intrinsic  12.9
4 intrinsic  13.6
5 intrinsic  16.6
6 intrinsic  17.2
```

We can then define a function to compute the standard error, which (in our case) measures the variability of the sample mean. From lecture, the formula is $SE = \frac{s}{\sqrt{n}}$ where s is the sample standard deviation and n is the number of observations. More correctly, it is the number of observations that are not missing values. The following approach defines a new function that computes the standard error given a vector (1 column) of numbers.

```
# function se(): calculates standard error
# x: the data
se <- function(x) {
  s <- sd(x, na.rm=TRUE) # computes sample standard deviation
  n <- sum(!is.na(x))     # computes number of non-missing observations
  s/sqrt(n)              # computes and returns standard error
}
```

If you are just getting started with R, we provide here some help understanding how to read (and eventually write) functions R. The following chunks breakdown the above function definition piece-by-piece. Starting with the outer shell of the expression:

```
se <- function(x) {
}
```

this starts the definition of a function that accepts one argument, `x`. Inside the function, whatever variable is passed into the function is called `x`. The contents of the function are stored in the object called `se`.

```
s <- sd(x, na.rm=T)
```

This computes the standard deviation of the values in `x`. the `na.rm=TRUE` bit tells R to ignore any missing values.

```
n <- sum(!is.na(x))
```

This counts the number of non-missing values. `is.na(x)` returns a `TRUE` value if the observation is missing and `FALSE` if not. `!` is the logical NOT operator. `TRUE` becomes `FALSE`, `FALSE` becomes `TRUE`. If you do arithmetic on a logical (`TRUE/FALSE`) value, `TRUE` is treated as a 1 and `FALSE` as a 0. So `sum(!is.na(x))` counts the number of non-missing values.

```
s/sqrt(n)
```

`sqrt()` computes the square root of a number (here `n`). So this computes the standard error of a mean by dividing the sample standard deviation by the square root of the number of observations. This is the evaluated expression, so it's returned as the result.

Now that we've defined `se()`, we can call it just like any other function.

```
# calculate standard error for all 47 observations
# notice that we only pass in the column of interest
se(creativity$score)
```

```
[1] 0.7653046
```

You can also implement this the tidyverse way if you wish:

```
# call se function within summarize, returns a data frame
creativity |>
  summarize(
    se=se(score)
  )
```

```
      se
1 0.7653046
```

We can also calculate standard error by treatment group (and this is where tidyverse becomes very helpful)

```
# compute se by group, returns a data frame
creativity |>
  group_by(treatment) |>
  summarize(
    se=se(score)
  )
```

```
# A tibble: 2 x 2
  treatment      se
  <chr>      <dbl>
1 extrinsic  1.10
2 intrinsic  0.906
```

Randomization Tests

Base R does not include a function that computes randomization test p-values. We provide here the R code to do that, with details on how to set it up and run it for various scenarios.

In this example, `creativity$score` contains the response variable; `creativity$treatment` contains the grouping variable. These are copied into two new variables, `response` and `group`, which are the variables used elsewhere in the code. For randomization tests, H_0 : no difference between intrinsic and extrinsic.

```
# set seed for reproducibility
set.seed(5101)

# randomization test
# calculate the observed differences in mean score
observed_diff <- creativity |>
  group_by(treatment) |>
  summarize(mean_score=mean(score)) |>
  summarize(diff=diff(mean_score)) |>
  pull(diff)

# run randomization
nsim <- 10000 # number of simulations to run
# you will set this value you design your test.
# replicate simplifies loops when collecting repeated scalar results
simulated_diffs <- replicate(nsim, {
  # everything in between the braces { } is evaluated nsim times
  # pipe the creativity data frame into the mutate function
  creativity |>
    # create a new column of shuffled treatment labels
    # the sample(x) generates a random permutation of the elements of x
    mutate(shuffled=sample(treatment)) |>
    # group by the permuted treatment labels
    group_by(shuffled) |>
    # calculate the mean of each treatment group
    summarize(mean_score=mean(score)) |>
    # calculate the difference between the means of each treatment group
    summarize(diff=diff(mean_score)) |>
    # pull the difference
    pull(diff)
})
```

```
# calculate two-sided p-value
p_value <- (1+sum(abs(simulated_diffs)>=abs(observed_diff)))/(nsim+1)
p_value
```

```
[1] 0.00489951
```

`observed_diff` is the actual, observed difference in means between the first group and any other observations. The core of the code is a loop, executed once for each random assignment of labels to observations. `sample(treatment)` permutes the group labels. We then compute the mean difference for each group, calculate the difference, and store the results in vector `simulated_diffs`. The two-sided p-value is then the number of more extreme random differences, with +1 to include the observed sample.

Self Assessment

Exercise

Read in the tomato data, `tomato.csv`. These are the values used in the lecture notes (module 1, segment 7). This is a randomized experiment comparing a proposed “better” fertilizer to my usual fertilizer. Seven plants (treatment a) were randomly assigned to the “better” fertilizer; the other seven plants received the usual fertilizer. The measurement units are pounds of ripe tomato.

- Draw side-by-side box plots or dot-plots of the yield for each of the two fertilizers
- Estimate the difference in mean yield between the two fertilizers (a-b). Include units.
- Estimate the difference in median yield between the two fertilizers (a-b). Include units.
- Use a randomization test to test the null hypothesis of no effect of the fertilizer on mean yield. Report the two-sided p-value

Solutions

- difference in mean yield between the two fertilizers: 6.53 pounds (average for a: 8.16, for b: 1.63)
- Difference in median yield between the two fertilizers: 4.64 pounds (sample median for a: 5.66, for b: 1.02)
- Run the function provided for the randomization test and extract the required p-value

```

# set seed for reproducibility
set.seed(5101)

# read in the data
tomato <- read.csv("tomato.csv", header=TRUE)

# perform the randomization test
# calculate the observed differences in mean score
observed_diff <- tomato |>
  group_by(treatment) |>
  summarize(mean_yield=mean(yield)) |>
  summarize(diff=diff(mean_yield)) |>
  pull(diff)

# run randomization
nsim <- 10000 # number of simulations to run
# replicate simplifies loops when collecting repeated scalar results
simulated_diffs <- replicate(nsim, {
  # everything in between the braces { } is evaluated nsim times
  # pipe the creativity data frame into the mutate function
  tomato |>
    # create a new column of shuffled treatment labels
    # the sample(x) generates a random permutation of the elements of x
    mutate(shuffled=sample(treatment)) |>
    # group by the permuted treatment labels
    group_by(shuffled) |>
    # calculate the mean of each treatment group
    summarize(mean_yield=mean(yield)) |>
    # calculate the difference between the means of each treatment group
    summarize(diff=diff(mean_yield)) |>
    # pull the difference
    pull(diff)
})

# calculate two-sided p-value
p_value <- (1+sum(abs(simulated_diffs)>=abs(observed_diff)))/(nsim+1)
p_value

```

```
[1] 0.03659634
```